

Terminal - polecenia

Zadanie 1

Ustawić wartość parametrów wykorzystując polecenie rosparam set.

Nazwa parametru	Wartość
/description	"opis symulacji"
/goal/tolerance/x	0.03
/goal/tolerance/y	0.01

In [4]:

In [5]:

In [6]:

Zadanie 2

Ustawić wartość parametrów wykorzystując polecenie rosparam set. Jako wartość należy przekazać odpowiedni słownik.

Nazwa parametru	Wartość
/goal/tolerance/x	0.05
/goal/tolerance/y	0.02

In [8]:

Nazwa parametru	Wartość
/robots/robot3/id	1
/robots/robot3/tryb	"auto"
/robots/robot3/isActive	True

In [11]:

Zadanie 3

Utworzyć plik z danymi .yaml i je zaimportować.

Nazwa parametru	Wartość
/simulation_name	"test 1"
/robots/sim_robot/v	1.2
/robots/sim_robot/w	0.3
/robots/sim_robot/type	"medium_transporter"
/robots/sim_robot2/v	1.2
/robots/sim_robot2/w	0.3
/robots/sim_robot2/type	"heavy_transporter"

In [13]:

Zadanie 4

Zapisać wszystkie ustawione parametry do pliku.

In [16]:

Zadanie 5

Zapisać wszystkie parametry z przestrzeni nazw **robots** do pliku **robots.yaml**.

In [17]:

```
dumping namespace [/robots] to file [robots.yaml]
/robots/robot3/id=1
/robots/robot3/tryb=auto
/robots/robot3/isActive=True
/robots/sim_robot/v=1.2
/robots/sim_robot/w=0.3
/robots/sim_robot/type=medium_transporter
/robots/sim_robot2/v=1.2
```

Zadanie 6

Usunąć parametr simulation_name.

In [19]:

Zadanie 7

Usunąć przestrzeń nazw dla robots/sim_robot

In [20]:

```
deleted parameter [/robots/sim_robot]
```

Python

In [22]:

```
import rospy
import rosparam
```

Zadanie 8

Ustawić z użyciem funkcji z biblioteki rospy.set_param wartości parametrów tolerancji dojazdu do punktu z użyciem jednego polecenia.

Nazwa parametru	Wartość
/goal_tolerance/position	0.1
/goal_tolerance/orientaiton	0.03

In [23]:

Zadanie 9

Ustawić z użyciem funkcji z biblioteki rosparam.set_param wartości parametrów tolerancji dojazdu do punktu z użyciem jednego polecenia.

Nazwa parametru	Wartość
/goal_tolerance/position	0.1
/goal_tolerance/orientaiton	0.03

In [24]:

Zadanie 10

Sprawdzić czy parametr /robots/sim_robot/max_vel istnieje.

In [25]:

Out[25]: False

Zadanie 11

Usunąć parametry dla robots/robot3.

In [27]:

Python - rozbudowa klasy

Kod z tematu 3_Serwisy z miejscami rozbudowy o dodatkowe funkcjonalności.

Zadania do wykonania

1. Dopisać przy inicjalizacji atrybuty klasy z ustawionymi domyślnymi wartościami. Parametry:

- maksymalna prędkość postępowania
- maksymalna prędkość obrotowa
- tolerancja odległości na osiągnięcie celu

2. Uzupełnić funkcję **set_default_params** odpowiedzialną za ustawienie w serwerze parametrów domyślnych wartości dla atrybutów klasy utworzonych w pkt. 1. Parametry powinny być ustawione w przestrzeni nazw utworzonego robota.

3. Uzupełnić funkcję **remove_params** odpowiedzialną za usunięcie wszystkich ustawionych nazw parametrów z przestrzeni nazw tego robota.

4. Uzupełnić funkcję **update_params**, która dla zdefiniowanych parametrów w pkt. 1 aktualizuje wartości atrybutów na podstawie odczytu wartości parametrów z serwera parametrów. Jeśli jakiś parametr został uzniony i nie można go odczytać należy wyświetlić informację, że "Nie udało się zaktualizować wszystkich parametrów. Jeden z parametrów nie istnieje."

5. Uzupełnić metodę **start()** i **stop()** odpowiednio o ustawienie/usunięcie parametrów robota w serwerze parametrów.

6. Dodać parametry i obsługę pisaka dla robota. Należy dodać parametry od koloru pisaka (r,g,b, width, off) tylko do serwera parametrów. Do obsługi pisaka dodać metody klasy odpowiedzialne za włączenie pisaka w ustawionym kolorze lub jego wyłączenie (2 metody). Dodać odpowiednie importy.

Do testów można usunąć z symulacji domyślnego robota turtle1 i utworzyć obiekt robota o tej samej nazwie. Można uruchomić sterowanie z klawiatury.

Poleceniem `rosparam set` przestawić kolor pisaka robota. ustawić metoda klasową nowy kolor pisaka i ruszać dalej robotem po planszy.

```
In [1]: import rospy
from turtlesim.srv import TeleportAbsolute, Spawn
from std_srvs.srv import Empty
from pkg_tsr.srv import Task, TaskResponse
from turtlesim.srv import SetPen, SetPenRequest

rospy.init_node("unikalna_nazwa_noda_parameter_serwer")

class Robot:
    def __init__(self, name, x, y):
        self.name = name
        # utworzenie serwera /go_to_goal
        self.follow_service = rospy.Service(str(self.name) + '/go_to_goal', Task,
                                             self._go_to_goal)
        self.square_service = rospy.Service(str(self.name) + '/draw_square', Task,
                                             self._draw_square_function)
        # (6) dopisanie inicjalizacji klienta, który będzie wysyłał żądanie zmiany
        # koloru pisaka

        self.create_robot(x,y)
        # (1) TODO dopisanie atrybutów od stałych parametrów robota

        self.set_default_params()

    def set_default_params(self):
        # (2) TODO dopisanie ustawienia parametrów w serwerze parametrów dla robota

        # (6) TODO dopisanie kolejnych atrybutów od koloru pisaka
        pass

    def remove_params(self):
        # (3) TODO dopisanie usuwania wszystkich parametrów ustawionych dla tego robota
        pass

    def update_params(self):
        # (4) Aktualizacja atrybutów klasy na podstawie parametrów ustawionych w
        # serwerze parametrów
        pass

    def create_robot(self, x, y):
        # Użycie istniejącego klienta do utworzenia robota w symulacji
        create_new_turtle = rospy.ServiceProxy('spawn', Spawn)
        create_new_turtle(x, y, 0, self.name)

    def start(self):
        # Wznowienie działania serwisów dla robota.
        self.follow_service = rospy.Service(str(self.name) + '/go_to_goal', Task,
                                             self._go_to_goal)
        self.square_service = rospy.Service(str(self.name) + '/draw_square', Task,
                                             self._draw_square_function)
        # (5) TODO ustawić parametry robota na serwerze parametrów na podstawie wartości
        # w atrybutach klasy

    def stop(self):
        # Zatrzymanie aktywnych serwisów dla robota.
        self.follow_service.shutdown()
        self.square_service.shutdown()
        # (5) TODO usunięcie parametrów robota z przestrzeni nazw

    def _go_to_goal(self, msg_srv):
```

Tutaj powinien znaleźć się program obsługujący działanie serwera /go_to_goal. Po wysłaniu zapytania przez dowolnego klienta na ten serwer powoduje on, że robot realizuje zadanie dojazdu do punktu. W zależności od tego czy udało się zrealizować dojazd czy nie zwracany jest odpowiedni wynik. Dla kilku robotów nazwy serwerów powinny być związane z nazwami różnych robotów.

```
"""
# Utworzenie i uzupełnienie odpowiedzi odpowiednimi wartościami.
response = TaskResponse()

# zwrócenie wyniku
return response

def _draw_square_function(self, req):
    # req - przekazany argument jest typu pkg_tsr/TaskRequest,
    # req jest zapytaniem wysłanym przez klienta
    width = 5
    print("Init pose x=%s, y=%s" % (req.x, req.y))
    # utworzenie niezbędnych klientów do teleportacji żółwia i narysowania kwadratów
    set_pose = rospy.ServiceProxy('/turtle1/teleport_absolute', TeleportAbsolute)
    clear_map = rospy.ServiceProxy('clear', Empty)
    set_pose(req.x, req.y, 0)
    clear_map()
    set_pose(req.x, req.y + width, 0)
    set_pose(req.x + width, req.y + width, 0)
    set_pose(req.x + width, req.y, 0)
    set_pose(req.x, req.y, 0)

    # response - odpowiedź serwera na zapytanie klienta jest typu pkg_tsr/TaskResponse
    response = TaskResponse()
    response.result = "finished"
    return response
```

In [2]: